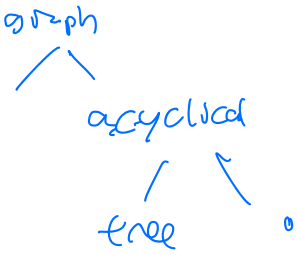


- tree is a graph
  - acyclical
  - n-1 edges & connected
  - parent → children

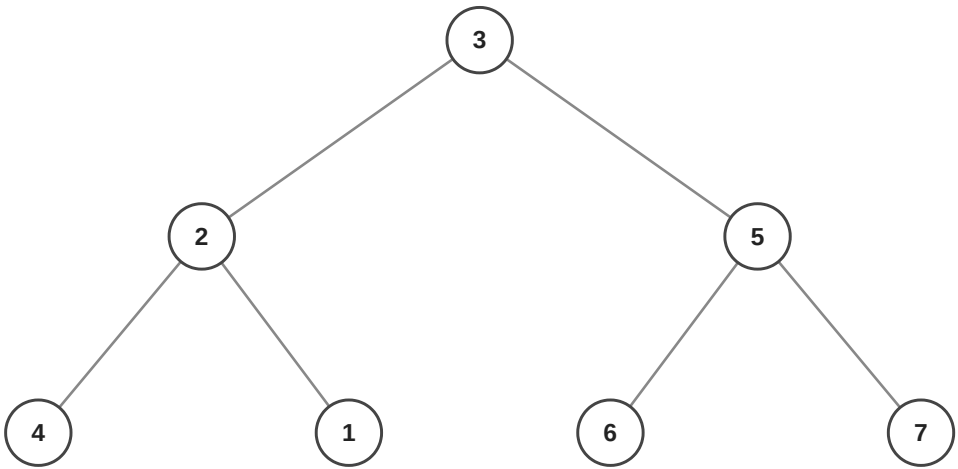


Dynamic programming on trees

class Node

Node[] children (can't be shared)

Problem: Sum of points



Sum of points

```
class TreeNode:
    def __init__(self, val, left, right):
        self.val = val
        self.left = left    # TreeNode or None
        self.right = right  # TreeNode or None
        self. dp_val = ... 0
```



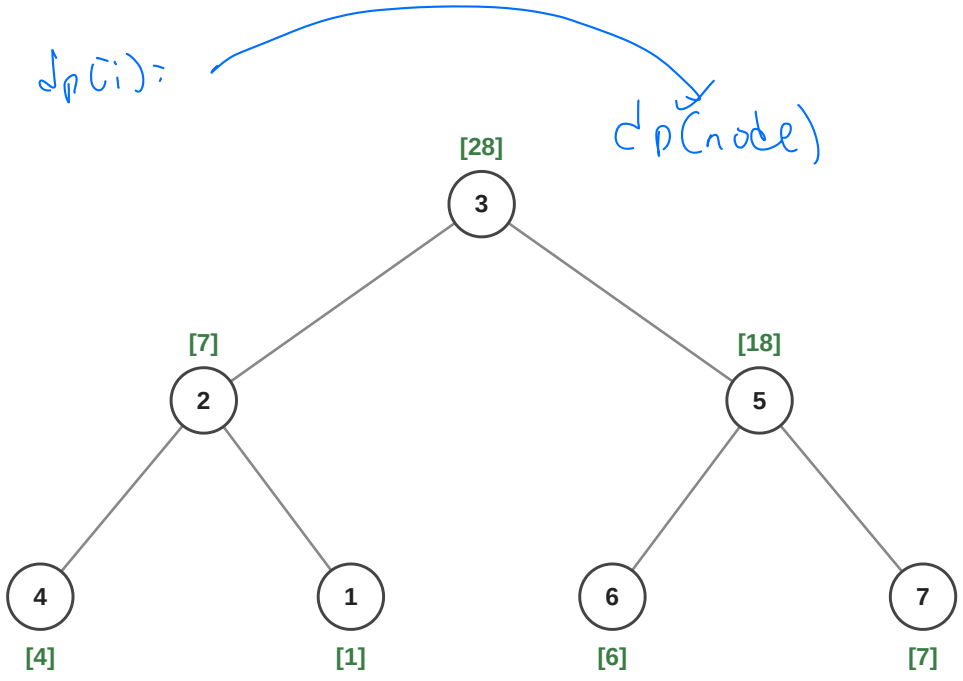
Input: `root`, a `TreeNode`.

Output: the sum of `val` over every node in the tree.

idea: ol. traverse → BFS / DFS

```
f(node):
    eval(value)
    f(l)
    f(r)
```

Step 2. Find recurrence



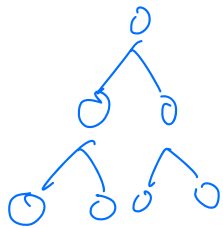
Step 2. Find recurrence

`dp(node)` = points in the subtree at `node`

```
dp(node) = node.val + dp(left) +
           dp(right)
```

```
dp(None) = 0
```

N nodes



O(n log n)?

Code

```
def total(node):
    if not node:
        return 0
    return node.val + total(node.left) + total(node.right)
```

Q: Runtime A: O(N)

Q: Why not V/E? A: O(V+E) = O(N+N-1) = O(N)

Q: Diff vs array A:

- no memo? → why? → rely only on immediate children
- dp(node): f(dp(child1) ...)

Arrays vs trees

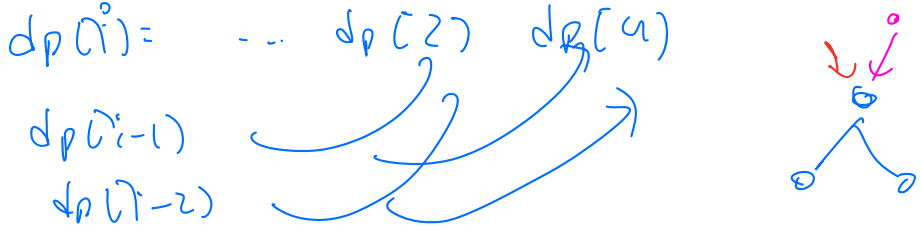
	Arrays	Trees
subproblem	`dp[i]`	`dp(node)`
recurrence	`dp[i] = f(dp[i-1], ...)`	`dp(node) = f(dp(left), dp(right))`
order	you pick the sweep direction	no choice -- just DFS
storage	a table you allocate	the return value

Why no dp table?

Every node has **exactly one parent**.

So `dp(node)` is called **exactly once** -- only one caller will ever want your answer.

The overlapping subproblems that *force* memoization on arrays never show up (*usually)



Old cookbook

1. **Define subproblems.** `dp[i]` = "best" way to handle nums[0..i]
2. **Find recurrence.** `dp[i] = f(dp[i-1], dp[i-2], ...)`
3. **Specify order.** `i = 0..n-1` or `i = n-1..0`?
4. **Base case.** `dp[0] = ?`

Cookbook, on trees

1. **Define subproblems**. `dp(node)` = "best" way to handle the subtree at `node`
2. **Find recurrence.** `dp(node) = f(dp(left), dp(right))`
3. ~~**Specify order.**~~ -- no order to pick. Just DFS: children first, then me.
4. **Base case.** `dp(None)` = ? -- and the missing children

Sum of points, by the cookbook

1. **Define subproblems.** dp(node) = total points of subtree rooted at node
2. **Find recurrence.** dp(node): node.val + dp(n.l) + dp(n.r)
3. ~~**Specify order.**~~ dfs
4. **Base case.** dp(null) = 0

Problem: Tree diameter

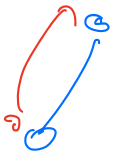
- path start & end on ~~leaves~~ / can't go down anymore.
- must root be used?

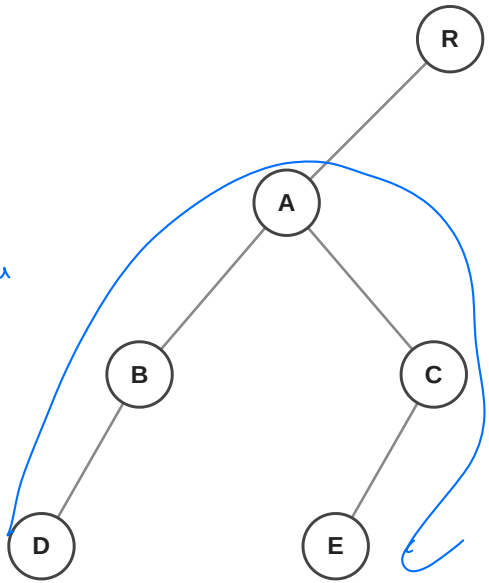
  no.
- intuition:

  dp(node): max depth of subtree

  case1: node is used

  case2: node is unused





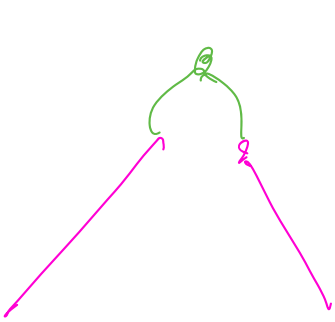
5 nodes

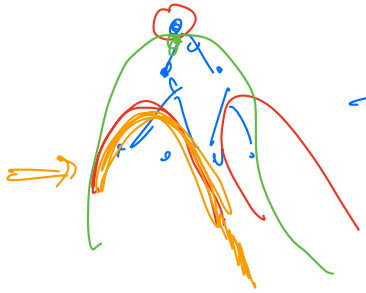
Tree diameter

Input: `root` of a binary tree.

Output: the length of the longest path between any two nodes, in **edges**.

Leetcode





→ hard to reuse :(

Step 1. Define subproblems

`longest_path(node) = ?` under this subtree

→ first good attempt

def. must use node

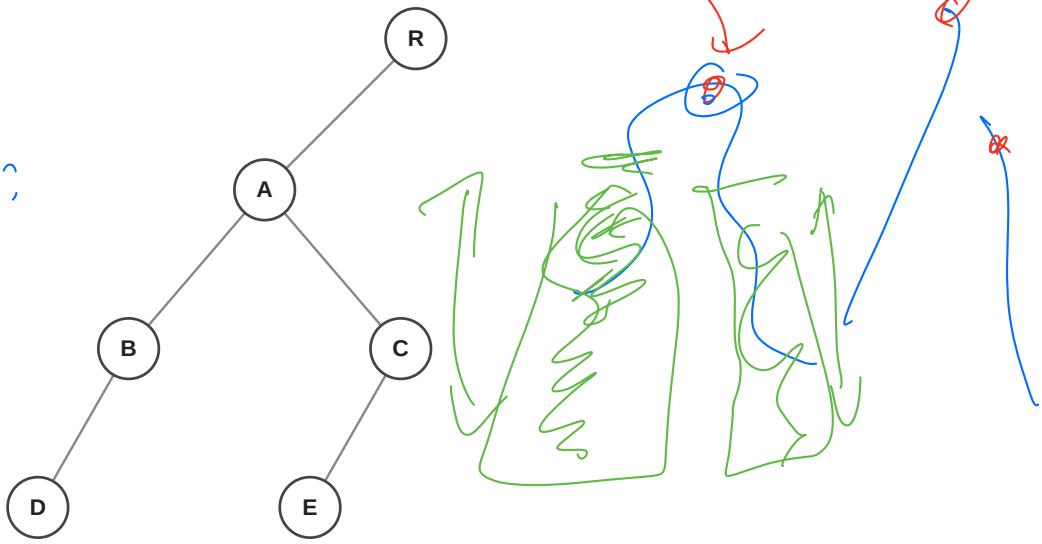
def 2: might not use node

idea:
- store longest ∩ shape
- store longest downward path /

Every path has a highest node

Q: If I tell you the peak node of the longest path.

how do you compute longest path?

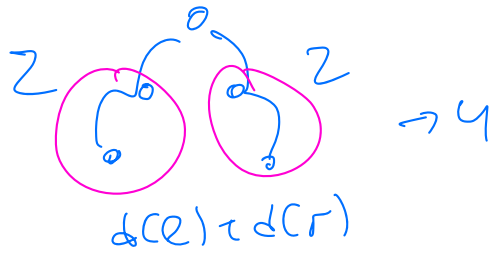


A: depth of left child + depth of right + 1

- take maxes over all. → store dp(node) → store highest (1 + d(l) + d(r)) → global max



Plan



`depth(node)` = height of the subtree at `node`, in nodes. `depth(None) = 0`

`through(node)` = longest path with `node` as its peak, in edges

answer = `max` of `through(node)` over every node

Code

```
longest_path = 0

def depth(node):
    global longest_path
    if not node:
        return 0
    L = depth(node.left)
    R = depth(node.right)
    longest_path = max(longest_path, L + R)
    return 1 + max(L, R)

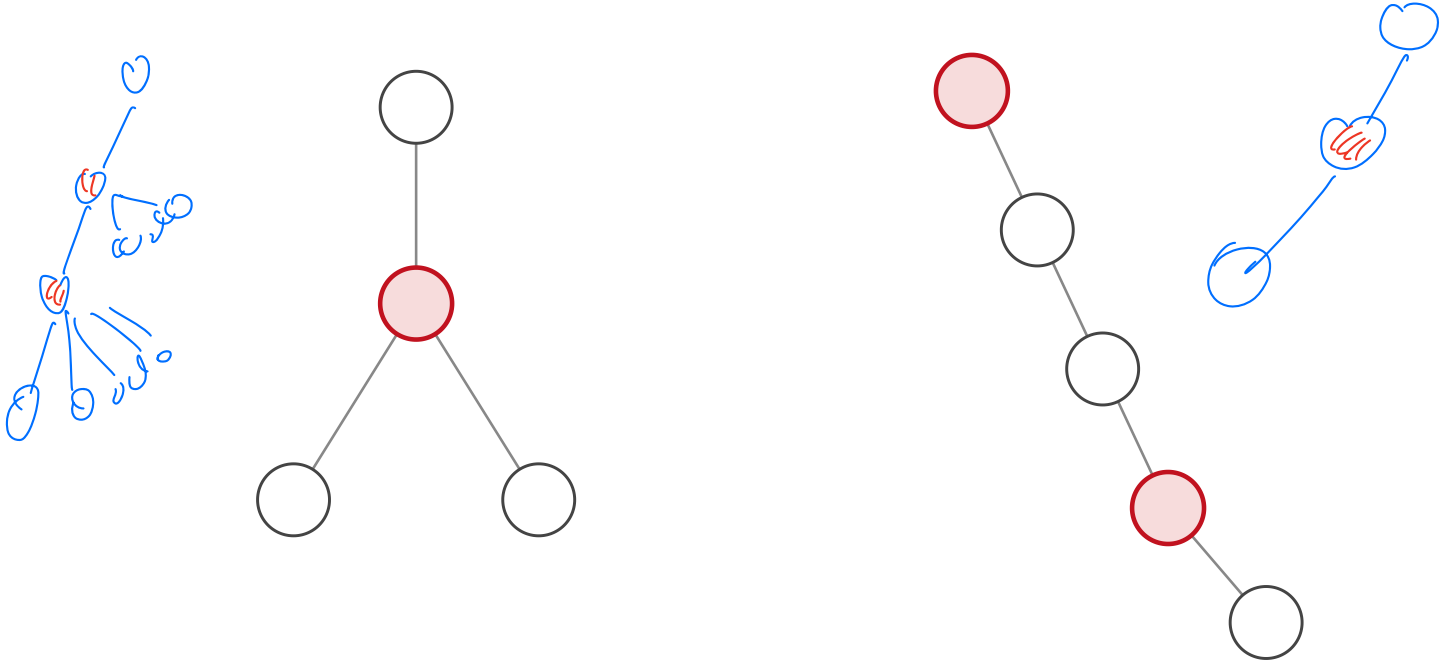
depth(root)
return longest_path
```

lesson: longest path (node) → dp_depth (node)

Q is always

need a different subproblem for dp

Problem: Binary tree camera



red = camera

Binary tree camera

Input: `root` of a binary tree.

Output: the minimum number of cameras so that **every** node is watched.

A camera at node v watches v, v's parent, and v's children.

Leetcode

Step 1. Define subproblems

dp(node) = ?

defn. min # cameras such that each node is always "watchable" for the subtree rooted at node

recurrence → dp(left) + dp(right) + 1 if neither left/right has camera

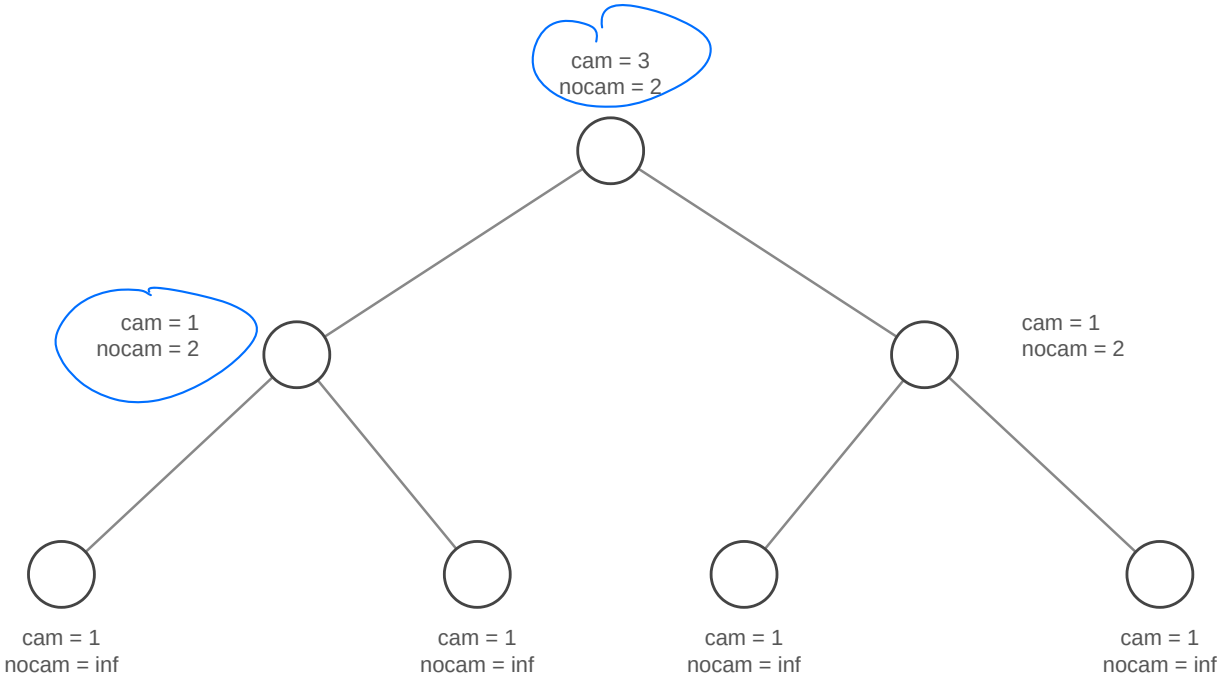
States!

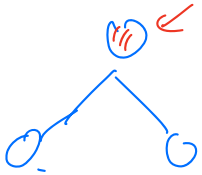
Step 1. Two states

`dp_camera(node)` = min cameras to cover the subtree at `node`, **given `node` has a camera**

`dp_nocam(node)` = min cameras to cover the subtree at `node`, **given `node` has no camera**

The two states, on an example





Step 2. Find recurrence

`dp_camera(node)` = 1 + ? for the left child + ? for the right child

I have a camera. My children do **not** need one.

So the left child costs... `dp_nocam(left)`?

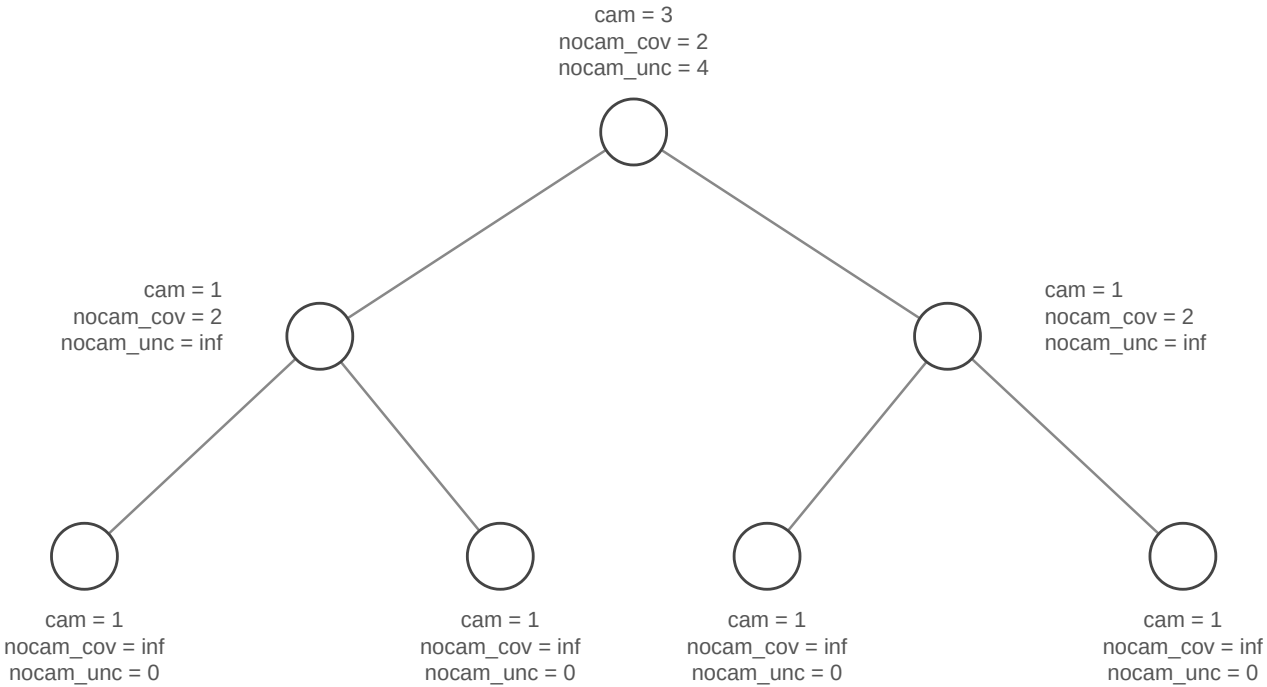
Step 1. Three states

`dp_camera(node)` -- camera at `node`

`dp_nocam_covered(node)` -- no camera at `node`, but a **child** watches it

`dp_nocam_uncovered(node)` -- no camera at `node`, **nobody below watches it**. My parent must.

The three states, same tree



Step 2. Camera here

```
dp_camera(node) =
  1
  + min( all three states of the left child )
  + min( all three states of the right child )
```

Step 2. Covered by a child

```
dp_nocam_covered(node) = min(
  dp_camera(left) + min( dp_camera(right), dp_nocam_covered(right) ),
  dp_camera(right) + min( dp_camera(left), dp_nocam_covered(left) )
)
```

Step 2. Nobody below

```
dp_nocam_uncovered(node) = dp_nocam_covered(left) + dp_nocam_covered(right)

dp(None) = ( camera = inf, nocam_covered = 0, nocam_uncovered = inf )
```

Code

```
def solve(node):
    if not node:
        return (INF, 0, INF)

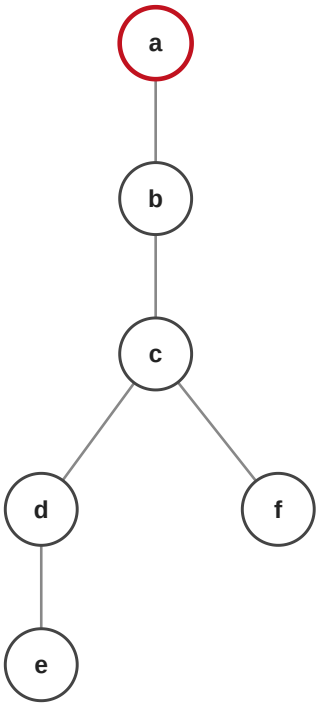
    lc, lcov, lunc = solve(node.left)
    rc, rcov, runc = solve(node.right)

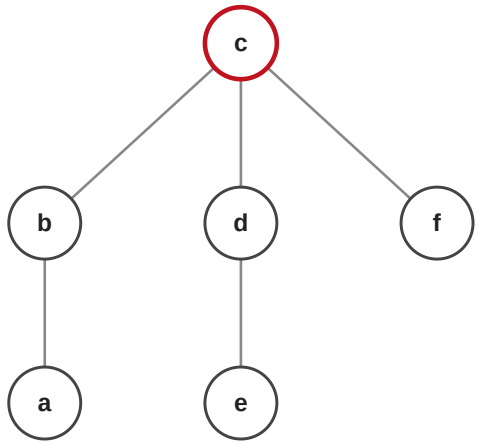
    cam   = 1 + min(lc, lcov, lunc) + min(rc, rcov, runc)
    cov   = min(lc + min(rc, rcov), rc + min(lc, lcov))
    unc   = lcov + rcov

    return (cam, cov, unc)
```

Lesson: dp w/ states for trees

Problem: Tree rerooting height





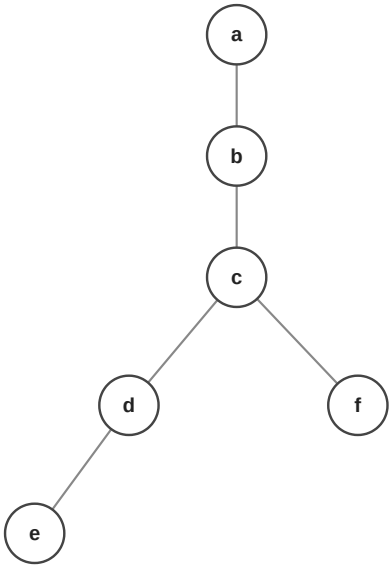
Tree rerooting height

Input: n, and an edge list of a tree (undirected, $n - 1$ edges, connected).

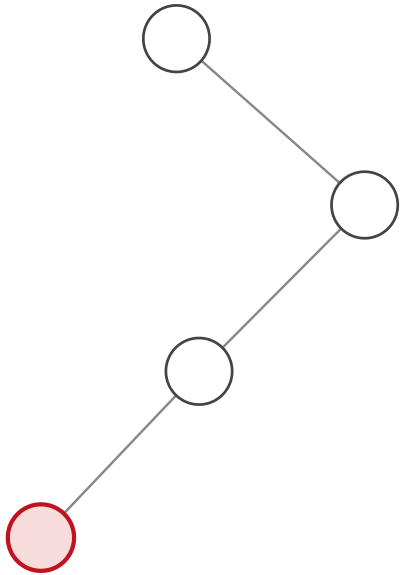
Output: for each node, the height of the tree rooted at that node.

(easier) Leetcode

Insight: Up or down



Insight: Longest up path



Plan so far

```
down(v) = height of the subtree at v, in nodes
  = 1 + max over children c of down(c)
  = 1 if v is a leaf

up(v) = longest path that leaves v going upward, in nodes
  = 1 + max( up(parent), sibling_branch, 1 )

  up(parent) -- keep climbing past my parent
  sibling_branch -- turn around at my parent, into a sibling
  1 -- stop at my parent

sibling_branch = 1 + max over parent's OTHER children s of down(s)

height rooted at v = max( down(v), up(v) ) - 1   # nodes -> edges
```

Insight: max, except me

Given an array, return for each position the max of **everything else**.

Insight: max, except me

Given an array, return for each position the max of **everything else**.

```
top1, top2 = the two largest values, duplicates kept

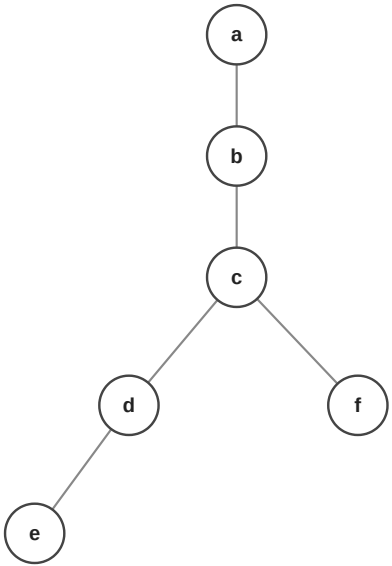
except_me(i) = top1 if a[i] != top1
except_me(i) = top2 if a[i] == top1
```

`[5, 5, 3]` -> `top1 = 5, top2 = 5` -> `except_me(0) = 5`

Plan

1. root the tree anywhere, say at node 0
2. down[v] -- post-order, children before me (bottom up)
3. up[v] -- pre-order, parent before me (top down)
4. height rooted at v = max(down[v], up[v]) - 1

Lesson: Look up



Recap: the cookbook

1. **Define subproblems**. `dp(node)` = "best" way to handle the subtree at `node`
2. **Find recurrence.** `dp(node) = f(dp(left), dp(right))`
3. ~~**Specify order.**~~ -- no order to pick. Just DFS: children first, then me.
4. **Base case.** `dp(None)` = ? -- and the missing children

Problem types

Simple. One value per node. Sum of points, node count, height.

Best path. Every path has a **highest** node -- brute force the peak. Tree diameter.

Multiple states. `dp1(node)`, `dp2(node)`, ... Binary tree camera.

Look above. You need the tree *above* you. Tree rerooting height.